Testing approaches, test plans and test data

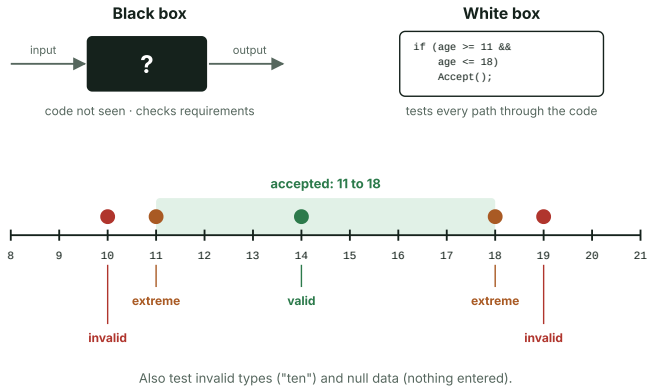
D22 FACT FILE

Specification: students should be able to

- Explain the following iterative approaches to testing: white box; black box; system, unit and integration testing
- Create and use a test plan that identifies test procedures for use during and after development to check a system against success criteria
- Devise and use the following types of test data: valid, invalid and extreme

CCEA fact file: Testing

Key words: iterative testing, white box, black box, unit testing, integration testing, system testing, test plan, test data, valid, invalid, extreme, boundary, null, expected outcome, actual outcome



Black box and white box testing, and test data on a number line for an 11 to 18 age range

Approaches

White box testing examines the internal logic so that every path and condition is executed; it is usually done by the programmer using dry runs and trace tables. Black box testing checks inputs against expected outputs without looking at the code, usually by independent testers or users. Unit testing checks one module on its own; integration testing checks modules work together; system testing checks the whole system against the specification. Testing is iterative – carried out repeatedly during and after development.

Test data and plans

Use valid data (should be accepted), extreme/boundary data (at the limits), invalid data (should be rejected) and null data (nothing entered). A test plan is a table showing each test's number, purpose, test data, type of data, expected outcome, actual outcome and any action taken.

Key terms from the mark schemes

White box testing
Testing the internal code, paths and logic, usually by the programmer.
Credited: U4 2019 Q10, 2024 Q8, 2025 Q9

Black box testing
Testing inputs and outputs against requirements without seeing the code.
Credited: U4 2019 Q10, 2024 Q8, 2025 Q9

Unit testing
Testing a single module, function or method on its own.
Credited: U4 2022 Q10, 2024 Q8

Integration testing
Testing that separately built units work together.
Credited: U4 2019 Q10, 2023 Q10

System testing
Testing the complete, integrated system against its specification.
Credited: U4 2019 Q10, 2022 Q10

Test plan
Table listing each test, its purpose, test data, expected outcome, actual outcome and action.
Credited: U4 2024 Q8, 2025 Q9

Valid / normal data
Typical data that should be accepted.
Credited: U4 2019 Q9

Extreme / boundary data
Values at the edges of the accepted range.
Credited: U4 2023 Q10, 2024 Q8

Invalid / erroneous data
Data that should be rejected.
Credited: U4 2023 Q10, 2024 Q8

Null data
No data entered.
Credited: U4 2019 Q10, 2024 Q8

Exam focus

2019 Q3 and Q9–Q10, 2022 Q2(c) and Q10(a), 2023 Q10, 2024 Q8, 2025 Q9. Learn clear definitions of each testing type and practise filling test plan rows.

Unit 4 2025 Q9	[11]	Test plan components; black vs white box
Unit 4 2024 Q8	[18]	String and Boolean types; unit testing; test plan; black vs white box
Unit 4 2023 Q10	[9]	Testing types gap fill; invalid and extreme test data
Unit 4 2022 Q10	[9]	Matching five testing methods; error trapping
Unit 4 2022 Q2	[14]	Data type for a race time; lowest of three algorithm; test data

? Check yourself

1. Describe two differences between black box and white box testing. [4]
2. Describe unit testing. [2]
3. Describe the difference between integration testing and system testing. [2]

Answers: 1. Two differences, each stated for both (2 marks each), e.g.: white box testing looks at the internal code, logic and structure (1) – black box testing looks only at inputs and outputs without seeing the code (1) / white box is usually done by the programmer (1) – black box by independent testers or end users (1) / white box aims to test every path, condition and line (e.g. with dry runs/trace tables) (1) – black box checks the functions meet the requirements using a test plan of inputs and expected outputs (1) 2. Testing one module/function/unit on its own (1) / done first, by the developer, before it is combined with other parts (1) 3. Integration testing checks that separately built units work correctly together (the interfaces between them) (1) / system testing checks the complete system against its specification/requirements, using the test plan (1)